



ATLANTIC COMPUTATIONAL EXCELLENCE NETWORK

A First Peek at Perl

Ross Dickson, Computational Research Consultant

What is Perl?

- Practical Extraction and Reporting Language
- General-purpose interpreted language
- Very popular, many applications
 - Web scripting
 - Systems administration and automation
 - ...many, many more
- Influenced by natural languages
- Greatest strength is text manipulation

Invocation

```
$ perl first.pl  
...  
$ head -1 first.pl  
#!/usr/bin/env perl  
$ chmod +x first.pl  
$ ./first.pl  
...
```

Examples 1

- first.pl
 - “my”, scalar and array variables, [index]
 - strings, single and double quotes, “\n”
 - backticks
 - chomp()
- hash1.pl
 - hashes or associative arrays, {key}
- math.pl
 - use Pkg::Pkg

first.pl

```
#!/usr/bin/env perl
use strict;
use warnings;

my $scalar = 'abcd';
my $number = 42;
print "scalar is $scalar, number is $number\n";
print 'but single quotes do not interpolate: $scalar, $number',
"\n";

my @list = ('a','b','c','d','e');
print "list is @list \n";
print "first element is $list[0] \n last element is $list[4] \n";

my @seq = `seq 1 10`;    # backticks to call out to shell
print @seq;
chomp(@seq);             # removes newline characters
print "chomped seq: @seq\n";
```

hash1.pl

```
#!/usr/bin/env perl
use strict;
use warnings;

my %table = (
    "I" => 1,
    "V" => 5,
    "X" => 10,
    "L" => 50,
    "C" => 100,
    "D" => 500,
    "M" => 1000,
);

print "V means $table{'V'}\n";
print "C means $table{C}\n";      # bareword, bad style!
```

math.pl

```
#!/usr/bin/perl -w
use strict;
use warnings;

print sqrt(2), "\n";

use Math::Trig;

print pi, "\n";
print cos(pi/2), "\n";

my $pi = 22/7;
print "my pi: $pi\n";
my $i = int($pi);
print "integer part: $i \n";
```

Examples 2

- passthru1.pl
 - @ARGV
 - filehandles
 - while, for, foreach, if, ...
 - \$_
- passthru2.pl
 - while (<>) idiom
- uc.pl
 - standard Perl library functions

passthru1.pl

```
#!/usr/bin/env perl
use strict;
use warnings;

print "$ARGV[0] \n";
open (INFILE, $ARGV[0]);
while (<INFILE>) {
    print $_;      # default variable for many things, aka $ARG
}
```

passthru2.pl

```
#!/usr/bin/perl -w
use strict;
use warnings;

while (<>) {      # idiom: Read all argument files, or stdin if
none
    print $_;
}
```

uc.pl

```
#!/usr/bin/perl -w
use strict;
use warnings;

while (<>) {
    print uc;      # uppercase function; if no argument, works on $_
}
```

Loops

- `while (condition) { statements; }`
- `until (condition) { statements; }`
- `for (expr; expr; expr) { statements; }`
- `foreach var (in_list) { statements; }`
- `do { statements; } while (condition)`
- `do { statements; } until (condition)`

Conditions

- Anything 0 or empty or undefined is false,
- anything else is true
 - ...more or less.
For more details, see the documentation.

Branches

- `if (condition)`
 `{ statements; }`
`elseif (condition)`
 `{ statements; }`
`else`
 `{ statements; }`
- `unless (condition) { statements; } . . .`
- `do { statements; } if (condition)`
- `do { statements; } unless (condition)`

Examples 3

- hash2.pl
 - foreach
 - sort()
 - keys()
- grab_geom.pl
 - “do X or die”
 - =~ /regular_expression/
 - split

hash2.pl

```
#!/usr/bin/env perl
use strict;
use warnings;

my %table = (
    "I" => 1,
    "V" => 5,
    "X" => 10,
    "L" => 50,
    "C" => 100,
    "D" => 500,
    "M" => 1000,
);

foreach my $key (sort keys %table) {
    print "$key\t$table{$key}\n"; }
```


grab_geom.pl

```
#!/usr/bin/env perl
# Extract "Optimized Parameters" from a Gaussian output file.
# Michelle Shaw, Feb. 23, 2010, adapted by Ross Dickson June 2010
use strict;
use warnings;

open IFILE, "<$ARGV[0]" or die "Unable to open $ARGV[0] for reading: $! \n";

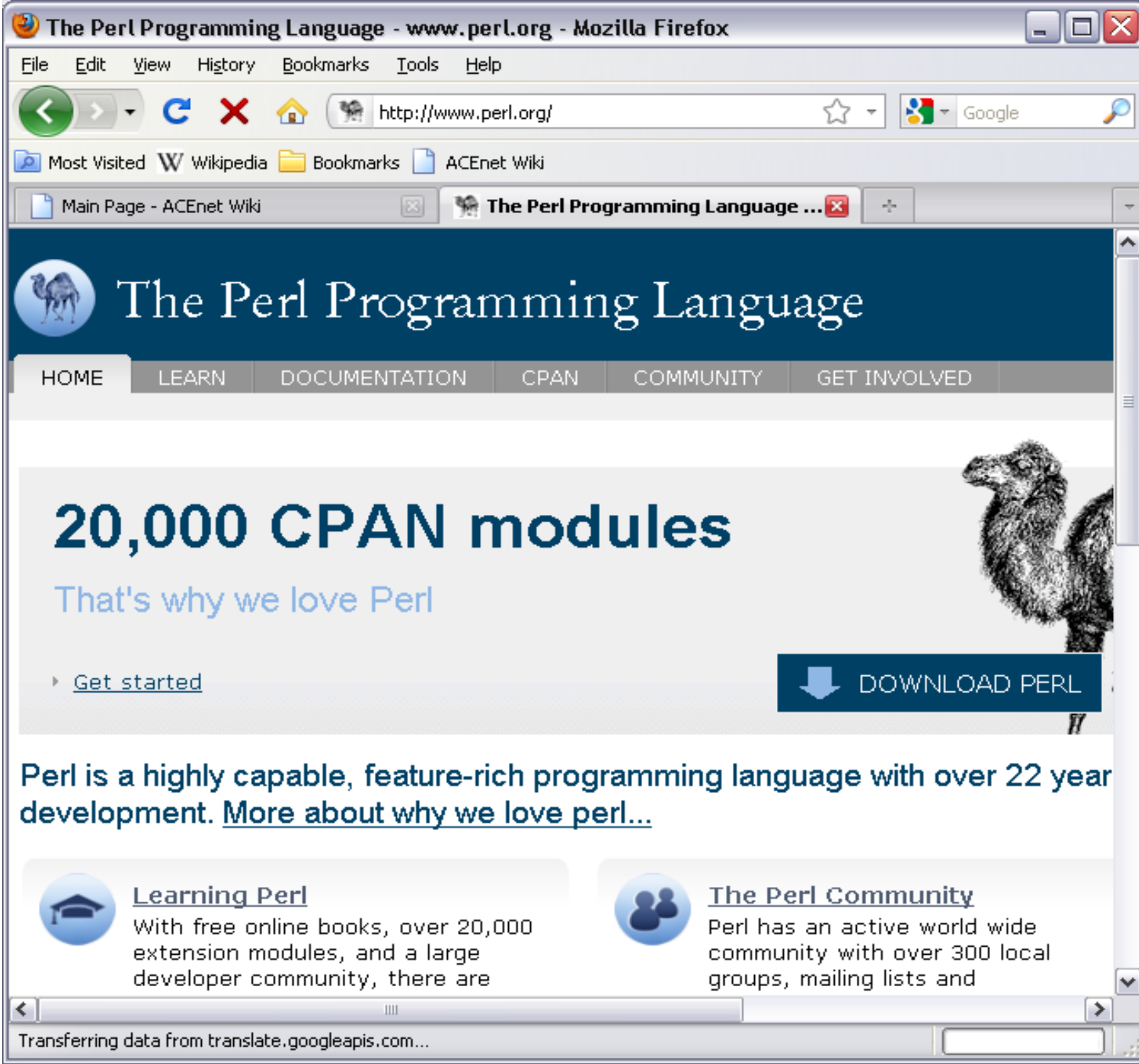
while (my $line = <IFILE>) {
    if ($line =~ /Optimized Parameters/) {

        # skip next four lines, inelegantly
        my $junkline = <IFILE>; $junkline = <IFILE>;
        $junkline = <IFILE>; $junkline = <IFILE>;

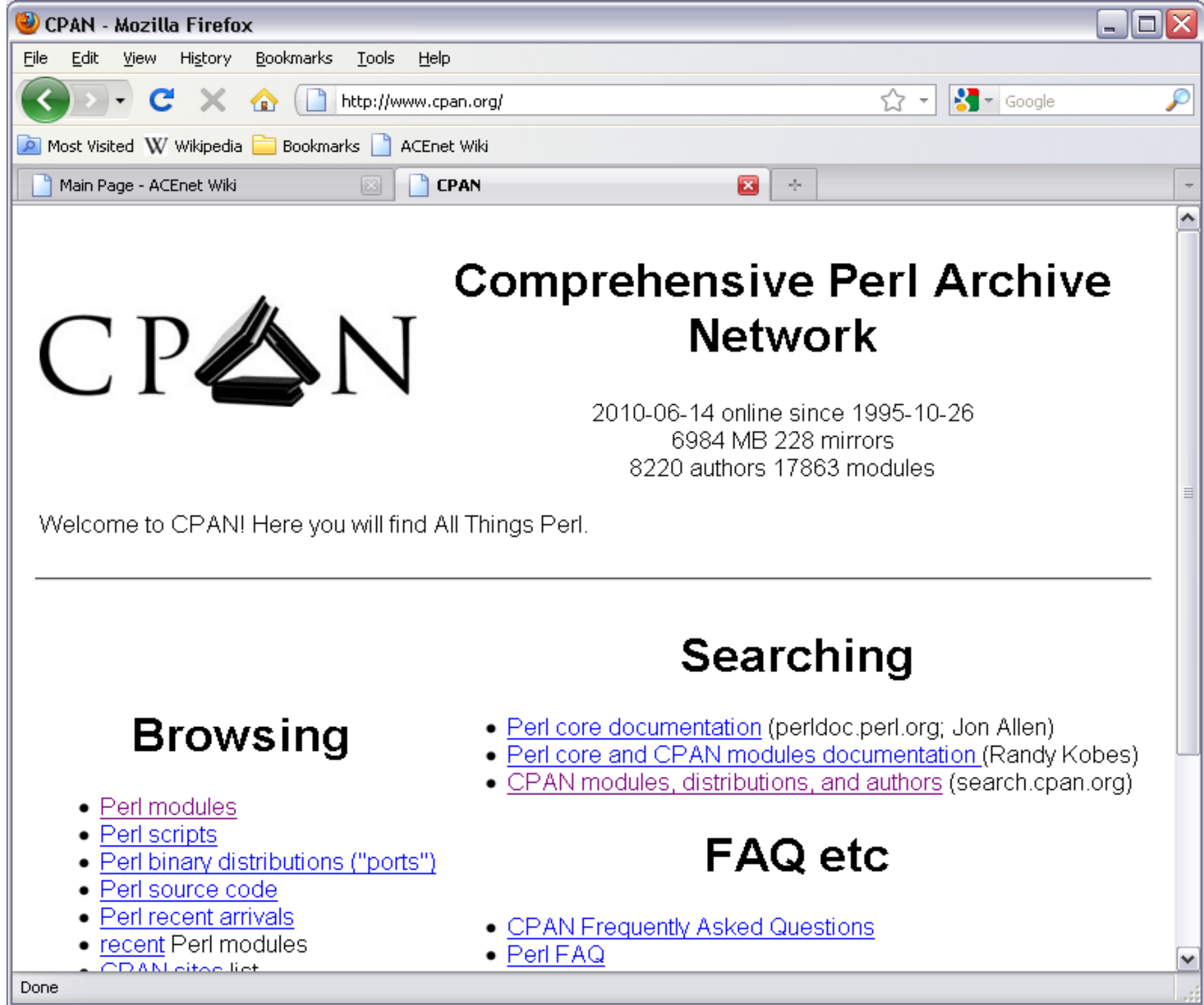
        $line = <IFILE>;
        while ($line =~ / ! /) {
            chomp($line);
            my @words = split ' ', $line;
            my $coord = $words[1];
            my $value = $words[2];
            print STDOUT "$coord $value\n";
            $line = <IFILE>;
        }
        print STDOUT "\n";
    }
}

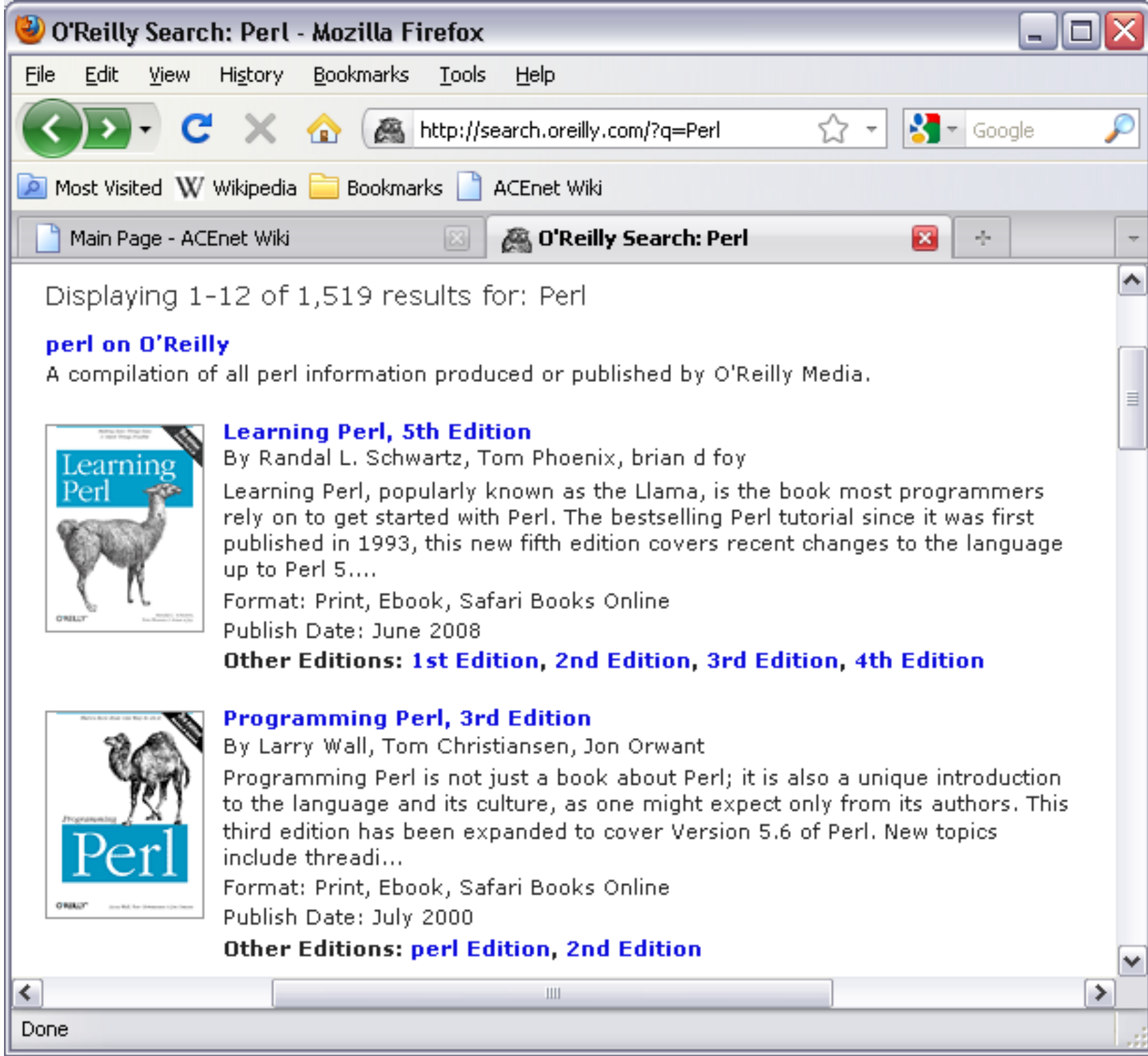
close IFILE;
```

http://www.perl.org



http://www.cpan.org





http://pdl.perl.org

Perl Data Language - Mozilla Firefox

File Edit View History Bookmarks Tools Help

http://pdl.perl.org/

Most Visited Wikipedia Bookmarks ACEnet Wiki

Perl Data Language - ACEnet Wiki Perl Data Language

PDL $\sum a_n \cos(nx) + b_n \sin(nx)$

perl> p sequence 5,5

```
[ 8 1 2 3 4]
[ 5 6 7 8 9]
[16 11 12 13 14]
[15 16 17 18 19]
[26 21 22 23 24]
```

Perl Data Language

Scientific computing with Perl

Quick links

- Home
- Download
- External Libs

Support

- Wiki
- Mailing list
- Documentation
- FAQ

Usage

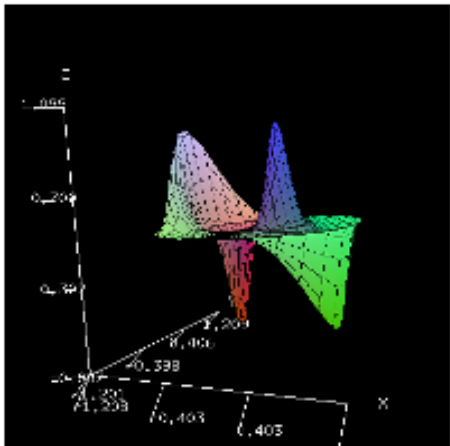
- Success Stories
- Screenshots
- Demos

Credits

- The Team
- Citation

Development

- Get Started



PDL ("Perl Data Language") gives standard Perl the ability to *compactly* store and *speedily* manipulate the large N-dimensional data arrays which are the bread and butter of scientific computing.

PDL turns Perl in to a free, array-oriented, numerical language similar to (but, we believe, better than) such commercial packages as IDL and MatLab. One can write simple perl expressions to manipulate entire numerical arrays all at once. For example, using PDL the Perl variable `$a` can hold a 1024x1024 floating point image, it only takes 4MB of memory to store it and expressions like `$a = sqrt($a) + 2` manipulate the whole image in a few milliseconds.

http://www.bioperl.org

BioPerl - Mozilla Firefox

File Edit View History Bookmarks Tools Help

http://www.bioperl.org/wiki/Main_Page

Most Visited W Wikipedia Bookmarks ACEnet Wiki

Perl - ACEnet Wiki BioPerl

Log in / create account

page discussion view source history

Main Page

Welcome to BioPerl, a community effort to produce Perl code which is useful in biology.

For more background on the BioPerl project please see the [History of BioPerl](#).

BioPerl is distributed under the [Perl Artistic License](#). For more information, see [licensing BioPerl](#).

main links

- [Main Page](#)
- [Getting Started](#)
- [Downloads](#)
- [Installation](#)
- [Recent changes](#)
- [Random page](#)

search

Go Search

documentation

- [Quick Start](#)
- [FAQ](#)
- [HOWTOs](#)
- [API Docs](#)
- [Scrapbook](#)
- [BioPerl Tutorial](#)
- [Tutorials](#)

OJBIF News

- [BioPerl has moved to GitHub](#)
- [OJBIF Google Summer of Code Accepted Students](#)
- [OJBIF in Google Summer of Code](#)
- [BioPerl at GMOD Meeting 2010](#)
- [Sanger FASTQ format and the Solexa/Illumina variants](#)
- [BioPerl interview in latest FLOSS Weekly](#)
- [BioPerl core 1.6.1 PPM available](#)
- [First 1.6.1 alphas of BioPerl-Run, BioPerl-DB, BioPerl-Network](#)
- [BioPerl 1.6.1 released](#)
- [Release 1.6 of BioPerl-run, BioPerl-db, BioPerl-network](#)

See also our [news page](#), and [twitter](#).

Done

Not to mention...

- subroutines and shift()
- references
- regular expressions and pattern matching operators
- comparison operators
 - ==, !=, <, >, <=, >= for numbers
 - eq, ne, lt, gt, le, ge for strings
- logical “short-circuit” operators
- operators: arithmetic, string, bitwise, file test
- scalar context vs. list context
- local scope vs. global scope

Example code

- On ACEnet clusters...
 - brasdor
 - courtenay
 - fundy
 - glooscap
 - placentia
- ...in `/home/rdickson/public/Perl_demo.tar`
 - `"tar xf /home/rdickson/public/Perl_demo.tar"`